HACK THE VALLEY 11 · SCHOOL OF CITIES · HOUSING AND TRANSIT READINESS

FutureProof Build Order

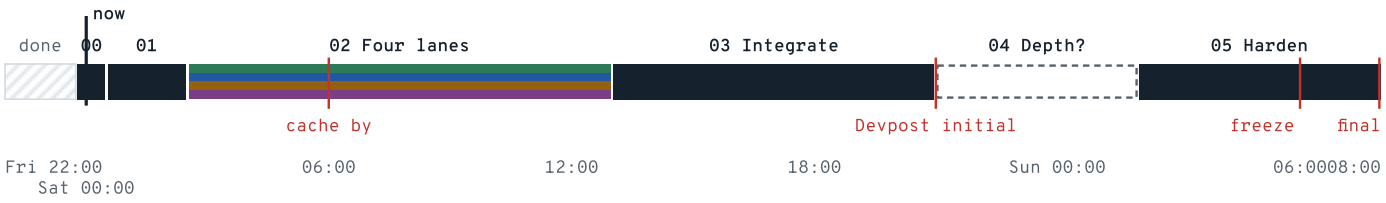
Every file in the proposed repo, mapped to an owner and a build step, from the final idea in the brainstorming doc. The order is a dependency chain: each phase unblocks the next, and the four lanes in Phase 2 only run in parallel because Phase 1 proved the data exists and froze the contract.

111 steps **4** owners **6** phases **1** corridor, 8–12 stops **3** developments · **2** phases
4 packages × **3** demand cases Fri 22:00 → Sun 08:00

A	**Owner 2**	Lane A · data, demand, corridor bundle
B	**Owner 3**	Lane B · simulation, verification
C	**Owner 4**	Lane C · API, Tiger Data, Gemini, packages
D	**Emilia**	Lane D · map, replay, comparison, demo

Owner numbers follow section 13 of the final idea. Click a name to change it. Tick steps as they land; names and ticks are saved in your browser only.

THE CLOCK · FRIDAY 22:00 KICKOFF TO SUNDAY 08:00 FINAL SUBMISSION



00 Foundation

All four together · Fri 23:45 → Sat 00:30 · 0 / 8 done

Nobody writes product code until all four laptops run the backend, the frontend and a database connection. Fixing one broken environment at hour 20 costs far more than fixing it together now.

ALL HANDS

Make it run on four laptops

Make it run on four laptops

- [ ] 0.1 **`requirements.txt`**

  Pin versions: `fastapi`, `uvicorn`, `pydantic`, `numpy`, `pandas`, `geopandas`, `shapely`, `pyproj`, `osmnx`, `psycopg[binary]`, `google-genai`, `python-dotenv`, `pytest`. Pinning now prevents the "works on mine" hour.

- [ ] 0.2 **`.gitignore`**

  Add `node_modules/`, `__pycache__/`, `.env`, `data/cache/*`, `data/raw/*`, `dist/` **before** the first `npm install`. The city massing download must never reach git.

- [ ] 0.3 **`.env.example`**

  `GEMINI_API_KEY`, `GEMINI_MODEL`, `DATABASE_URL` (Tiger Data), `DATABASE_URL_LOCAL` (fallback), `DATA_CACHE_DIR`, `SIM_RUNS`, `OFFLINE`. Keys stay on the backend.

- [ ] 0.4 **`backend/config.py`**

  One pydantic `Settings` object loading `.env`, plus `MODEL_VERSION` and `DATA_VERSION` strings that every saved result records.

- [ ] 0.5 **`docker-compose.yml`**

  Local PostgreSQL + PostGIS with the same schema as Tiger Data. If venue wifi drops, the demo switches database URL and keeps working.

- [ ] 0.6 **`frontend/`**

  `npm create vite@latest frontend -- --template react-ts`, then add `maplibre-gl` and one chart library. Owner 1 decides which and nobody adds a second.

- [ ] 0.7 **`scripts/setup.sh`**

  venv → `pip install -r requirements.txt` → `npm install` → database ping. One command per teammate.

- [ ] 0.8 **`README.md`**

  Ten lines: what FutureProof is, how to run the backend, worker and frontend. Expand at the end.

GATE — DO NOT PROCEED UNTIL

All four people have `uvicorn backend.main:app --reload`, `npm run dev` and `SELECT postgis_version();` working locally, and a first commit is pushed.

01 Prove feasibility, freeze the contract

All four together · Sat 00:30 → 02:30 · blocking · 0 / 16 done

The final idea says to choose the corridor only after inspecting real files, and to agree IDs, units, direction, coordinates, scenario fields, event types and result fields before splitting. Do both here. The simulation and data owners work side by side: a demand and model mismatch can invalidate two working lanes at once.

IN PAIRS · TIME-BOXED TO 75 MINUTES

Feasibility checks

- [ ] 1.1 `docs/corridor.md` DECISION

Pick one direction of one TTC bus corridor, 8–12 consecutive stops, and one weekday peak window. Two Scarborough candidates to inspect first:

Eglinton East–Kingston Rd–Morningside

Routes 86 / 116 / 905 / 986. 8.5 km of RapidTO bus lanes installed in fall 2020, ending at UTSC. Published before/after travel times give the lane option a local source. About 36,500 average weekday boardings in Nov 2025.

Watch: lanes already cover most of it, so the lane option is a gap, such as the 116's mixed-traffic stretch through Guildwood. Published savings also include stop removals and pandemic traffic.

Lawrence Avenue East

Routes 54 / 154 / 954. About 35,800 weekday boardings in Nov 2025. A 2025 City report lists RapidTO for Lawrence East among unfunded work, which makes "ready before occupancy" a live question.

Watch: no observed lane effect on this street. Any lane effect borrowed from Eglinton East is labelled as a transferred assumption.

Choose the first one where Owner 2 finds three documented residential projects within walking distance of the stops. Use the AM peak toward the subway: new residents board near home, so their load lands on existing riders downstream, which is the effect the brief asks you to show. Record stop IDs, direction, window, and what is left out (express trips, the other direction).

- [ ] 1.2 `docs/feasibility.md`

Confirm three developments exist in the Development Pipeline with unit counts and stages, and that the GTFS feed has the chosen pattern on a weekday service date. List

which demand observations are missing. If stop-level demand stays unavailable, the product is labelled a *scenario experiment* from here on.

- [ ] 1.3 `scripts/check_access.py`

  Tiger Data service reachable with PostGIS enabled; one Gemini call returns a valid structured response. If either fails, decide the fallback now, not at hour 20.

- [ ] 1.4 `sim/tiny_case.py` OWNER 3'S FIRST MILESTONE

  Three stops, two buses, ten passengers, worked out by hand first and written in the docstring. The simulator's first job is to reproduce that page exactly.

ALL HANDS · STRICT ORDER

Schemas, innermost to outermost

Each object below is composed by the next. Build them in this order and the last one assembles itself.

- [ ] 1.5 `backend/schemas/source.py`

  `SourceRef` : source_id, title, url, publisher, retrieved_at, version, licence. Every other object points at these ids.

- [ ] 1.6 `backend/schemas/assumption.py` CRITICAL

  `Assumption` : key, unit, meaning, low / central / high, allowed range, status (`sourced` | `assumed` | `user_selected`), source_ids. The UI shows unit, meaning, status and range on every control, and this is where they come from.

- [ ] 1.7 `backend/schemas/corridor.py`

  `Stop` (stop_id, gtfs_stop_id, name, seq, lon, lat), `Link` (link_id, from_stop, to_stop, length_m, base_time_s), `Corridor` (route pattern, direction, peak window, stops[], links[], berths per stop, incoming load at the first stop).

- [ ] 1.8 `backend/schemas/development.py`

  `Development` : dev_id, pipeline_ref, address, units, stage, record_date, kind (`proposal` | `hypothetical`), footprint, access[] (stop_id, walk_m), source_ids. Units are proposed; residents exist only after an occupancy assumption.

- [ ] 1.9 `backend/schemas/intervention.py`

  `Intervention` : id, type (`priority_lane` | `extra_service`), link_ids or headway, capital_cost and operating_cost ranges with price year and scope, extra vehicles, earliest_phase, effect with status.

- [ ] 1.10 **backend/schemas/scenario.py**

`Scenario` : scenario_id, version, developments per phase, occupied share, demand case, intervention availability dates, capital and operating budgets, fleet allowance, planning criterion. Housing totals are fixed across package comparisons.

- [ ] 1.11 **backend/schemas/run.py**

`Run` : run_id, scenario_version, package, model_version, data_version, seed ids, status, diagnostics (conservation, unresolved passengers, runtime).

- [ ] 1.12 **backend/schemas/replay_event.py**

`ReplayEvent` : run_id, t_s, kind (`dispatch | arrive | alight | board | deny | depart`), vehicle_id, stop_id, count, load_after, rider_group. This one type drives the moving buses, the queues and the replay stored in Tiger Data.

- [ ] 1.13 **backend/schemas/results.py**

`StopResult` by rider group (existing, new): mean and p95 wait, share denied at least once, remaining queue, headway variability. `CaseResult` adds per-run arrays, run count and unresolved passengers.

- [ ] 1.14 **backend/schemas/comparison.py** CONVERGENCE

`Comparison` : scenario, corridor, cases (current, growth, supported growth), packages[] with feasibility reasons and criterion results per phase and demand case, sensitivity, assumptions[], sources[], warnings[]. If a lane's output does not fit in here, that lane is building the wrong thing.

- [ ] 1.15 **frontend/src/lib/types.ts**

Hand-mirror 1.5 through 1.14 in TypeScript, with field names identical to the Python.

- [ ] 1.16 **data/fixtures/sample_comparison.json** DO NOT SKIP

One fully populated `Comparison` plus a short replay stream (two buses, every stop), written by hand. Set `"fixture": true` and show a FIXTURE banner whenever it is on, so made-up numbers never reach a judge.

GATE — DO NOT PROCEED UNTIL

The corridor is chosen from real files, the tiny case matches the hand calculation, `sample_comparison.json` validates against the pydantic models and the TypeScript types compile against it. Then split up.

02 Four lanes in parallel

Sat 02:30 → 13:00 · the long stretch · sleep in staggered pairs · 0 / 56 done

Each lane runs and tests on its own. Order matters within a lane, not across lanes. Two cross-lane waits, both stubbed until they land: B3 needs A7's walk table, and C5 needs B11's paired runner.

A

LANE A · OWNER 2

Corridor bundle, developments, demand inputs

Own what the model is allowed to believe. Every row carries a source id and retrieval date; anything you cannot source enters as an assumption with a status.

- [ ] A1 `data/sources/base.py`

  Download-once helper: on-disk cache, `retrieved_at` stamp, checksum. Every other source uses it.

- [ ] A2 `data/sources/gtfs_static.py`

  TTC Routes and Schedules: stops, trips, stop times and shape for the chosen pattern and service date. Resolve direction, branches, calendar exceptions and times after 24:00. Keep the feed's dates in its SourceRef.

- [ ] A3 `data/reviewed/developments.csv` CRITICAL

  Shortlist five Development Pipeline projects, keep the three best documented: source id, units, location, stage, record date. Check that related applications are not counted twice. Occupancy dates and completion shares are scenario inputs unless sourced.

- [ ] A4 `backend/db/schema.sql`

  Tables for sources, developments, stops, links, walking access, interventions, scenario versions, runs, results and replay events. PostGIS geometry where spatial; enable only extensions you use. Same file on Tiger Data and the local fallback.

- [ ] A5 `data/pipelines/build_corridor.py`

  Clip the stops, build links from the shape, compute lengths in metres (UTM 17N, EPSG:26917) and store display geometry in EPSG:4326. Assert stop order and direction match GTFS.

- [ ] A6 `data/pipelines/massing.py`

  3D Massing clipped to a few hundred metres around the corridor, simplified, exported

as GeoJSON with height. Record its vintage. Never ship the city-wide file to the browser.

- [ ] A7 `data/pipelines/walk_access.py` UNBLOCKS B3

OSMnx walk network for the supported area, cached with attribution. Walking distance from each development to its 2–3 nearest usable stops. Check three routes on the map for railways, barriers and missing crossings.

- [ ] A8 `data/reviewed/assumptions.yaml` CRITICAL

The demand chain, one factor per line with its denominator: occupied share → people per unit → peak trips → transit share → departure profile → destinations. Never multiply a per-person trip rate by a separate traveller fraction that counts the same thing twice.

- [ ] A9 `data/reviewed/background.yaml`

Existing riders by stop and the load already on board at the first modelled stop. TTC route-level boardings bound the total; write down the rule that spreads it across stops and the window. Matching a daily total does not validate that rule.

- [ ] A10 `data/reviewed/interventions.csv`

Two options. Priority lane: link_ids, effect range from RapidTO material or a labelled assumption, capital cost with price year, earliest phase. Extra service: new headway, round-trip cycle with layover, cost per vehicle-hour, service days. A similar project's budget is not a quote; mark it as such.

- [ ] A11 `scripts/record_vehicles.py` OPTIONAL

Check the TTC BusTime real-time feed's access terms and fields. If it works, record corridor vehicle positions for a held-out headway check. It is weekend service, not the weekday peak, and occupancy fields may be empty.

- [ ] A12 `scripts/seed_data.py` BY SAT 06:00

One run loads the whole bundle into Tiger Data and the local cache. From here the app never calls an upstream site.

- [ ] A13 `tests/data/test_bundle.py`

Every row has source ids, every assumption has a status and range, units are positive, walk distances are finite and plausible.

B LANE B · OWNER 3

Passenger and vehicle simulation, verification

Passenger and vehicle simulation, verification

Write the invariants before the clever parts. A simulator that loses passengers still draws a convincing animation. Bunching has to come out of the events, never out of a rule that forces it.

- [ ] B1 `sim/demand/journeys.py`

  Generate destination-consistent journeys (origin stop, destination stop or "beyond segment", arrival time) from the A8 chain, each tagged existing or new. Never invent boardings and alightings separately.

- [ ] B2 `sim/demand/streams.py`

  Separate random streams for arrivals, link noise and a shared corridor delay, pre-drawn per run so a change in event order cannot shift later draws. The same draws feed every package.

- [ ] B3 `sim/demand/stop_assignment.py` WAITS ON A7

  New residents to stops by walking distance. Use straight-line distance as a labelled stub until A7 lands.

- [ ] B4 `sim/engine/events.py · state.py`

  Event heap with stable tie-breaking; state for vehicles, loads and FIFO stop queues.

- [ ] B5 `sim/engine/stop.py`

  Berth rule (one bus serves a stop at a time unless the corridor says otherwise), no-overtaking rule, alight then board up to capacity, the rest stay queued.

- [ ] B6 `sim/engine/dwell.py`

  An explicit dwell rule from door time, boardings and alightings. This feedback makes a late bus later and lets the next one catch up.

- [ ] B7 `sim/engine/links.py`

  Link time = base + shared corridor delay + link noise. The priority lane changes only the parameters of its listed links.

- [ ] B8 `sim/engine/dispatch.py`

  Arrivals at the first modelled stop come from the GTFS schedule with their incoming load. Extra service is an alternative timetable. A faster lane does not add trips by itself.

- [ ] B9 `sim/engine/simulator.py`

  Warm-up before the window, a measured cohort defined by arrival time, and enough later service to observe its waits. Anyone still waiting at the analysis cap is reported as

unresolved, with a lower-bound wait.

- [ ] **B10** `tests/sim/test_invariants.py` BEFORE B11

  Every passenger is waiting, on board, arrived or beyond the segment. Load stays between zero and capacity. Denied riders persist and can board later, and are counted once. Event times ordered, travel times non-negative. A light regular case matches the hand result. An overloaded case grows a queue. Same inputs and seeds reproduce the stored result.

- [ ] **B11** `sim/runner/paired_runs.py`

  Start at 20 paired runs per case. Time the full load early: 20 runs × 4 packages × 3 demand cases × 2 phases is 480 runs before baselines and delay cases. Show run count and run-to-run variation.

- [ ] **B12** `sim/metrics/`

  Mean and p95 wait, share denied at least once, remaining queue, headway variability, by stop and rider group. A passenger wait percentile is not a confidence interval; label it that way.

- [ ] **B13** `sim/calibration/consistency.py`

  Baseline run times against the schedule, totals against route boardings, and A11's recorded headways as held-out data if it exists. Report the residuals. Never tune the growth case.

- [ ] **B14** `notebooks/`

  Scratch space. Promote stable code into the modules above; the notebook never becomes the system.

C LANE C · OWNER 4

Analysis API, storage, packages, grounded AI

The model computes; Gemini translates and explains. Every number on screen comes from Lane B's runs or Lane A's sources.

- [ ] **C1** `backend/main.py · api/deps.py`

  FastAPI app, CORS for the Vite port (5173), `/health` that also pings the database.

- [ ] **C2** `backend/services/validate.py`

  Every site, option, value and constraint checked against the reviewed bundle before analysis. Errors say what is wrong and the allowed range.

- [ ] C3 `packages/feasibility.py`

  Four packages: existing, lane, extra service, both. Vehicles = ceil(cycle ÷ headway). Operating cost = extra vehicle-hours × service days × cost per hour. Reject anything over capital, operating or fleet limits, or unavailable in the phase, and keep the reason.

- [ ] C4 `backend/worker/ · jobs table`

  A separate worker process runs simulations so web requests never block. A job table and polling are enough; add streaming only if the workload needs it.

- [ ] C5 `packages/evaluate.py` WAITS ON B11

  Run every feasible package on the same demand cases and draws. Until B11 lands, a stub returns fixture results.

- [ ] C6 `packages/compare.py`

  Mark dominated options. Name the least-cost tested package that meets the user's criterion across the selected cases, or show the alternatives when none does. Wording: "best among the tested packages under these assumptions."

- [ ] C7 `packages/sensitivity.py`

  Low, central and high demand plus one upgrade-delay case. Report whether the preferred package changes and which stops are repeatedly worst.

- [ ] C8 `backend/db/queries.sql` TIGER DATA STORY

  The three queries to show judges: development access joined to affected stops, worst stops for a phase, and one package compared across runs and assumption versions. Store aggregates for all runs and replay events for selected runs only. Measure before claiming any speed-up.

- [ ] C9 `backend/api/routes/`

  `corridor · scenarios · jobs · results · replay`, each returning only the frozen schemas, with source and scenario ids on every result.

- [ ] C10 `backend/core/cache.py`

  Key on the full validated input, data version, model version and random configuration. Editing an input marks earlier results stale.

- [ ] C11 `backend/ai/gemini_client.py`

  One model, structured output, key on the server, timeout, every call logged. If Gemini fails, the forms still work.

- [ ] C12 `backend/ai/scenario_parser.py`

  "Keep the same homes, bring the first phase forward, compare the affordable options" becomes a scenario patch using existing ids only, then goes through C2. Unsupported questions and missing observations get a plain answer saying so.

- [ ] C13 `backend/ai/explainer.py` DO NOT FAKE

  Explains a stored `Comparison` from its JSON alone. After generation, check that every number in the text appears in the payload and every source id exists; otherwise fall back to a templated explanation. If the comparison asked about doesn't exist, offer to run it.

- [ ] C14 `backend/services/brief.py`

  Text planning brief: compared options, costs, service outcomes, uncertainty, limitations, source versions, model version and seed ids.

- [ ] C15 `tests/ai/test_parser_cases.py`

  Ten fixed questions re-run after every prompt change: valid parse, correct ids, agreement with numeric results.

D LANE D · EMILIA

Map, replay, comparison, accessibility

Build the whole interface against `sample_comparison.json` from the first hour. Be feature-complete before the backend returns a real run, then one flag switches it live.

- [ ] D1 `frontend/src/lib/api.ts`

  Fixture first, real base URL behind `VITE_USE_FIXTURE`. Flipping it in Phase 3 is the integration.

- [ ] D2 `frontend/src/styles/tokens.css`

  Type scale, palette, phase colours and status styles that use shape or text as well as colour. Set once.

- [ ] D3 `src/map/CorridorMap.tsx · MassingLayer.ts`

  MapLibre with existing buildings as quiet extrusions. Existing, actual proposals and hypothetical alternatives look clearly different; simplified masses never look like an approved design. Keep every layer in one module so walk mode (4.1) can reuse it unchanged.

- [ ] D4 `src/panels/DevelopmentPanel.tsx`

- [ ] D4 `src/panels/DevelopmentPanel.tsx`

  Units, planning status, location, source link and the date of the source record.

- [ ] D5 `src/panels/StopPanel.tsx`

  Background demand, added development demand, service and results for the selected stop.

- [ ] D6 `src/views/ListView.tsx`

  The 2D list alternative with the same essential information as the map. Build it early; it is also the fallback if WebGL misbehaves on the demo laptop.

- [ ] D7 `src/panels/ScenarioEditor.tsx`

  Developments per phase, occupied share, demand factors, availability dates, budgets. Each control shows unit, meaning, status and range. The planning-phase timeline and the simulation clock are separate controls.

- [ ] D8 `src/replay/` DEMO SPINE

  Buses moving and queues changing, from replay events. Inspect a bus (capacity, aboard, recent boarding and alighting, times) or a stop (waiting, denied). Pause, speed, scrub. Counts on screen match the model. Labelled "one illustrative run".

- [ ] D9 `src/panels/CaseCompare.tsx`

  Current, growth and supported growth side by side: mean and p95 wait, share denied, remaining queue, headway variability, split by existing and new riders, with capital cost, operating cost and extra vehicles beside them.

- [ ] D10 `src/panels/PackageTable.tsx`

  Four packages with feasibility reasons, and whether each meets the user's criterion in each phase and demand case. A failed target is shown, never hidden.

- [ ] D11 `src/panels/Sensitivity.tsx`

  Low, central, high demand and the delay case: does the preferred package change, and which stops are worst every time.

- [ ] D12 `src/panels/Evidence.tsx` DIFFERENTIATOR

  Every number links to the inputs behind it: the assumption with its status, and a source link that opens the real City or TTC page.

- [ ] D13 `src/panels/AskBox.tsx`

  Gemini question → interpreted fields shown and editable → Run. Nothing changes

without the user seeing what changed. Stale results are marked.

- [ ] D14 `accessibility pass`

  Keyboard for every control, visible focus, text alternatives for charts, a pause for motion, advanced assumptions in an expandable panel.

GATE — DO NOT PROCEED UNTIL

Each lane works alone: the bundle is in Tiger Data; the simulator passes B10 and returns paired baseline results; packages are ranked against the stub; the interface plays the fixture with moving buses and queues. Four working parts, not yet connected.

---

03 Integration

All four, one table · Sat 13:00 → 21:00 · Devpost initial due 21:00 · 0 / 7 done

The brief says integrate before polishing. Wire the request flow in one direction and add routes only as the code beneath them works: load corridor → edit or parse scenario → validate → job → simulate feasible packages → check invariants → aggregate → store → render → explain and export.

SEQUENCED · NOT PARALLEL

Connect the chain

- [ ] 3.1 `backend/services/pipeline.py`

  The whole chain as one function. Run `python -m backend.services.pipeline --scenario demo` headlessly before any HTTP is involved.

- [ ] 3.2 `routes in order`

  corridor → scenarios → jobs → results → replay. Each returns real data before the next is written.

- [ ] 3.3 `backend/core/cache.py` DEMO INSURANCE

  Precompute what the judge can reach: 2 phases × 3 demand cases × on-time or delayed upgrade, all four packages each. The live challenge then returns a real stored result, shown as a cache hit.

- [ ] 3.4 `frontend/src/lib/api.ts`

Turn `VITE_USE_FIXTURE` off. If Phase 1 held, nothing else in the frontend changes.

- [ ] 3.5 **`tests/test_pipeline.py`**

  Demo scenario end to end: invariants pass in every case, the fixture flag is off, every assumption in the response has a status, an over-budget package is rejected. Run before every commit from here.

- [ ] 3.6 **reload test**

  On the deployed flow: reload, edit an input, confirm old results go stale and new ones replace them.

- [ ] 3.7 **Devpost initial submission** SAT 21:00

  Required for prize eligibility: name, thirty-second pitch, School of Cities as lead, team, repo link, one screenshot. It stays editable until the final deadline.

GATE — DO NOT PROCEED UNTIL

A user can load the corridor, change a housing phase, compare resource-feasible packages, inspect consistent stop-level results and uncertainty, and export a reproducible brief. An infeasible budget and an unresolved queue are reported correctly. **This is the MVP.** Tag the commit.

---

04 Walk mode, then stretch

Whoever is free · conditional, skip entirely if Phase 3 is not green · Sat 21:00 → Sun 02:00 · 0 / 15 done

If behind, cut stretch models, extra map layers and elaborate animation first. Each item below is worth doing only with the full loop already working.

D LANE D · EMILIA · FIRST STRETCH, ≈ 4–6 HOURS

Walk mode: the corridor at street level

A ground-level view of the same scene the map already draws: same massing, same developments, same replay. It is built on the MapLibre camera first because that reuses every layer at no extra cost. A separate Three.js scene with free WASD walking is a later option, only if the lite version is done. No AI-edited Street View photos: Google's terms restrict altering its imagery, and a photoreal edit would look like an approved design.

- [ ] 4.1 `src/map/WalkMode.ts` DEMO MOMENT

  A "Walk the street" button drops the MapLibre camera to eye height (pitch near 85°) at a chosen stop. W/S move along the street, A/D strafe, mouse drag turns. Movement is clamped to the corridor and the camera never goes inside a building footprint. Esc returns to map mode.

- [ ] 4.2 `src/map/WalkMode.ts · PhaseToggle`

  Toggle Today / Phase 1 / Phase 2 to raise or remove the proposed masses. Label phases, not calendar years: the brief treats occupancy dates as scenario inputs. Proposed buildings keep the "simplified, not an approved design" styling from D3.

- [ ] 4.3 `src/map/WalkMode.ts · stop scene`

  Stand at a stop while the replay runs: buses pull in and the queue marker grows, from the same events and counts as map mode. Toggle the growth phase and watch people get left behind. This ties the 3D view to the simulation.

- [ ] 4.4 `src/panels/DevelopmentPanel.tsx`

  "See this spot today" link that opens Google Maps Street View at the site's coordinates in a new tab. Real photos for reference with no API key and no licensing risk.

D SECOND PERSON ON LANE D · HIGH RISK · ≈ 12–20 HOURS

Full street world and future photos

A free-roam 3D street you can walk anywhere along the corridor, plus photo views showing the proposed buildings in place. This needs one person working on it full time from Phase 2, because it will not fit in the evening after integration. It never blocks the core loop: if it is not smooth by Sun 02:00, the demo uses walk mode lite (4.1–4.4).

- [ ] 4.5 `data/pipelines/street_geometry.py`

  OSM roads, sidewalks and crossings for about 1 km around the corridor, as flat polygons with widths from tags or stated defaults. Exported in local metres around a fixed corridor origin, the same origin the buildings use.

- [ ] 4.6 `src/street/StreetScene.ts`

  Three.js scene with a first-person camera: pointer-lock mouse look, WASD, eye height, collision against building footprints. Loads the 4.5 geometry and the same massing GeoJSON as the map. Enter from the map's "Walk the street" button at the camera's location.

- [ ] 4.7 `src/street/Buildings.ts`

Extruded massing with a procedural window-grid facade scaled to height, so it reads as a street without real textures. Proposed buildings use a distinct translucent material and follow the shared phase toggle (Today / Phase 1 / Phase 2).

- [ ] 4.8 `src/street/Transit.ts`

  Buses and stop queues driven by the same replay events as map mode, so the street view and the charts can never disagree.

- [ ] 4.9 `data/photos/ · src/street/FuturePhoto.tsx` NOT GOOGLE

  Before/after slider at 3–5 points on the corridor. Use photos the team takes on the street (the venue is next to the corridor's end) or Mapillary images under their licence with attribution, not Google Street View, whose terms restrict altering its imagery. Render the proposed massing from a camera matched to each photo (position, heading, field of view) and overlay it. An AI step may blend lighting only, and every image is stamped "Illustration — not an approved design".

- [ ] 4.10 `performance check · Sun 02:00`

  Smooth on the demo laptop with the replay running. If not, ship walk mode lite and show the photo slider only. Decide at 02:00, not at the demo.

THEN PICK FROM THE TOP · STOP WHEN TIME RUNS OUT

Depth that survives questions

- [ ] 4.11 `deploy/vultr/` C

  Host the API and worker on Vultr if you have access, and show a scenario computed there. No GPU needed.

- [ ] 4.12 `sim/engine/signals.py` B

  Signal priority with signal delay separated from other link delay, so lane and signal benefits are not counted twice.

- [ ] 4.13 `sim/engine/holding.py` B

  A validated headway-management option, labelled a model scenario, not TTC practice.

- [ ] 4.14 `data/reviewed/hypothetical_sites.csv` A · D

  Same-total housing alternatives on a few reviewed hypothetical sites. Moving a building on screen does not make the site developable; say so in the UI.

- [ ] 4.15 `backend/research/investigator.py` PAPIERS · OPTIONAL

Bottleneck investigator: hypothesis → controlled experiment on matched seeds → evaluation → next hypothesis, with Papiers read and write. Minimum: one full investigation with an agent-chosen next step. Findings describe the model, not the TTC.

05 Demo hardening

All four · Sun 02:00 → 08:00 · non-negotiable · 0 / 9 done

Four minutes to present. A rehearsed demo of 80% of the system beats an unrehearsed demo of all of it. The doc lists conflicting event times, so check the organizers' current notices tonight and move these times if needed.

ALL HANDS

Make it survive contact with judges

- [ ] 5.1 `scripts/run_demo.sh`

  One command: check Tiger Data (fall back to local), confirm the cached grid, start API, worker and frontend, open the corridor. Run it from a cold boot at least once.

- [ ] 5.2 `docs/demo_script.md`

  Names against each segment. 0:00–0:30 the decision · 0:30–1:15 the consequence (one stop, one replay, wait distribution, share left behind) · 1:15–2:15 the response (packages, housing held constant, costs) · 2:15–3:15 the judge delays the upgrade or raises demand · 3:15–4:00 the evidence chain and the brief. Rehearse three times.

- [ ] 5.3 `docs/qa_card.md`

  Answers for passenger conservation, incoming loads, how the lane effect was chosen, why extra service costs vehicles, and why the result differs from an average-capacity sum. Keep the hand-worked tiny case from 1.4 on it.

- [ ] 5.4 `usability check`

  Someone outside the team finds which tested package supports the selected phase within budget and explains why. Record completion, time and any misread uncertainty.

- [ ] 5.5 `OFFLINE=1`

  Everything served from cache and the local database with Gemini off. Test with wifi physically off.

- [ ] 5.6 `recorded backup`

  Screen recording of the full demo path after the second rehearsal, labelled as recorded.

- [ ] 5.7 `README.md · docs/assumptions.md · docs/architecture.md`

  Screenshots, the request-flow diagram, the assumptions table with statuses, and the limits: no confirmed stop-level demand data, schedules are not observed headways, transport outcomes are not emission claims.

- [ ] 5.8 `feature freeze` SUN 06:00

  Bug fixes only. Everyone commits and pulls; one laptop becomes the demo laptop and stops changing.

- [ ] 5.9 `Devpost final` SUN 08:00

  Lead with School of Cities; Climate Action as the theme; AI for Social Impact if the Gemini interaction works. Select Gemini, Tiger Data or Vultr only for integrations you actually demonstrate, and confirm each category's eligibility.

Standing rules

True in every phase

Eight rules that hold from hour zero. Most exist because breaking them is silent: you find out during the demo, or a judge finds out for you.

Cache to disk by Sat 06:00

GTFS, the massing clip, the walk network, development records and any recordings. The demo never waits on venue wifi.

Every number has a status

Sourced, assumed or user-selected, visible next to the value. Unsourced numbers ship as labelled assumptions, never as facts.

The model computes, Gemini translates

Gemini fills scenario fields from existing ids and explains stored results. It never produces passenger counts, costs, effects or permissions.

Hold housing constant

Packages are compared on the same homes and demand. A failed target never quietly removes homes or demand.

Matched draws for every case

Same seeds and streams across baseline, growth and every package, or the difference you show is noise.

Show the bad mornings

p95 wait, share left behind and unresolved passengers sit beside every mean. The replay is one run: the comparison is all of them.

Frontend never waits on backend

The fixture from 1.16 keeps Lane D at full speed. If the interface is blocked on an endpoint, Phase 1 went wrong.

Report what the model returns

"Best among the tested packages under these assumptions." If the preferred package never changes, say so. No claims about emissions, real waits or corridor collapse.

Built from the *final idea* tab of the Chuds brainstorming doc (FutureProof: Housing Growth and Transit Readiness), mapped onto a proposed repo layout. Event times come from the doc's Hack the Valley info section; confirm them against current organizer notices. Corridor figures checked 16 Oct 2026:

- TTC: RapidTO Eglinton East bus lanes
- City of Toronto and TTC: RapidTO dashboard
- TTC: 2025 operating statistics, boardings by corridor
- City of Toronto: 2025 RapidTO report (unfunded corridors)
- Steve Munro: RapidTO travel times, 2020–2022
- Toronto Open Data: TTC Routes and Schedules